RoboTrader Remediation and Launch Plan

RoboTrader project

RoboTrader Remediation and Launch Plan

Document date: 2026-07-20 Status: Active execution plan Source baseline: repository audit at commit 51f0e99 on branch main Target: safe supervised paper operation first, then remote read-only access, then an explicitly gated limited live canary

Current execution baseline (2026-07-23): main includes the truthful Phase 0 CI gates from PR #83 (7f5de0a) and runtime-stability fixes from PR #81 (b7e5005). PR #82 is the active PR 1 containment change.

1. Purpose

This document turns the full codebase audit into an ordered implementation program. It is intended to remain the reference plan for future pull requests, reviews, test campaigns, launch decisions, and operational sign-off.

The ordering is deliberate. Later work must not begin merely because earlier code was merged. Every phase has evidence gates that must pass in the actual supported runtime.

The plan addresses:

- trading correctness;
- IBKR and exchange integration;
- order execution and paper/live separation;
- risk, sizing, stops, take-profit, and strategies;
- backtesting and market-data quality;
- database integrity, backups, and reconciliation;
- security, secrets, identity, authorization, and API safety;
- logging, auditability, testing, CI/CD, packaging, and deployment;
- dashboard, mobile, cloud, missing features, and unfinished features.

2. Non-negotiable rules

1. Keep IBKR ReadOnlyApi=yes until the live-enablement gate in PR 11 has been implemented and independently approved.
2. Never delete or rewrite user trading data without explicit user authorization, a verified backup, a preview of the exact impact, and a tested rollback path.
3. Do not treat a green unit-test count as launch evidence. Launch evidence includes reconciliation, restart, restore, failure-injection, alert, and soak results.

4. Broker state becomes authoritative whenever real broker orders are possible. Local database state alone must never authorize trading after startup or reconnect.
5. A kill switch must block risk-increasing orders while preserving a narrowly defined reduce-only path.
6. No remote control, mobile control, or cloud exposure is allowed before identity, authorization, TLS, WebSocket privilege separation, and durable audit controls exist.
7. Backtests cannot approve a strategy until the accounting and execution model in PR 6 is complete.
8. Every production claim must name the supported topology. Until changed by an approved architecture decision, that topology is local macOS, IBC, IB Gateway paper account, and the authoritative launcher.
9. Every PR must be independently reviewable, revertible where practical, and leave the paper system in a coherent state.
10. Incomplete strategies and alternate engines remain disabled or quarantined rather than partially integrated.

3. Current baseline and containment boundary

The current sanctioned runtime is materially contained:

- START_TRADER.sh uses paper port 4002 and validates IBC read-only configuration.
- runner_async.py constructs PaperExecutor.
- The active subprocess broker client is read-oriented and does not expose a complete order-placement interface.
- The current local dashboard is bound to loopback.
- The preflight gate, launchd watchdog, persistent connection recovery, test isolation, and persisted kill switch are valuable foundations.

This containment must be made explicit and tested in PR 1. It must not be weakened while the later live-order architecture is built.

4. How to use this plan

For each PR:

1. Create one tracking issue using the PR section below as its body.
2. Confirm its prerequisites are complete and evidenced.
3. Add or update tests before changing safety behavior where feasible.
4. Implement only the stated scope. Record extra discoveries in the problem register rather than silently expanding the PR.
5. Run the PR-specific verification suite plus the repository regression suite.
6. Attach evidence to the PR: commands, test output, screenshots where applicable, migration/restore results, and fault-injection results.
7. Update this document's progress register after merge.
8. Do not advance a launch gate until every required PR and operational exercise for that gate is complete.

Recommended branch naming: codex/pr-XX-short-name or the team's equivalent feature prefix.

5. Dependency sequence

The required order is:

1. PR 1 preserves the safe paper-only boundary and removes destructive hazards.
2. PRs 2 through 5 make active paper behavior safe and state authoritative.
3. PR 6 repairs the evidence engine used to evaluate strategies.
4. PR 7 unifies strategy and risk behavior on top of corrected data and state.
5. PRs 8 through 10 establish remote security, reproducible delivery, and honest operations.
6. PRs 11 and 12 add live order placement and broker-native protection behind a disabled gate.
7. PR 13 proves failure behavior through soak and fault injection.
8. PR 14 delivers an explicitly supported remote/cloud topology if still required.
9. PR 15 permits only a tiny, manually armed live canary.

PRs may be prepared in parallel only when their files and safety contracts do not overlap. Merge order still follows the dependency chain.

Phase A - Preserve containment and correct the active paper system

PR 1 - Freeze live mode and quarantine destructive utilities

Objective

Make the current paper/read-only boundary unambiguous, machine-enforced, and visible. Remove the possibility that a dormant script can destroy user data during later remediation work.

Problems addressed

- Conflicting EXECUTION_MODE, TRADING_MODE, TRADING_ENV, and deployment-mode variables.
- Production manifests that claim live operation while the runner remains paper.
- Missing IBC-config branch that can allow startup checks to pass when the file is absent.
- Dashboard status that can report a hardcoded or incorrect mode.
- Destructive recovery and IB position-sync scripts.
- Shared paper/live database, logs, artifacts, or credentials.

Step-by-step work

1. Define one canonical runtime contract containing environment, execution mode, broker port, broker account identifier, broker account type, database path, model-artifact set, and build identifier.
2. Add a startup validator that rejects every inconsistent combination, including:
    - paper mode with live port 4001;
    - live mode with paper port 4002;
    - missing IBC configuration;
    - IBC ReadOnlyApi not equal to yes during the containment phase;
    - missing or unapproved broker account identifier;
    - shared paper/live database path;
    - production mode without authentication, signing, alerting, or backup readiness.
3. Make the authoritative launcher and runner refuse live execution regardless of environment values. Use a separate compile-time or capability flag that remains disabled until PR 11.
4. Replace dashboard hardcoded mode reporting with the validated server-side runtime contract.
5. Add an unmistakable PAPER banner containing account alias, execution source, database identity, build SHA, and configuration fingerprint.
6. Inventory every alternate launcher. Disable or route scripts/start_runner.sh, Docker commands, dashboard start, and Kubernetes commands through the same contract.
7. Quarantine simple_recover.py, recover_database.py, and sync_ib_positions.py so they cannot run accidentally.
8. Design replacement maintenance commands with:
    - read-only preview;
    - portfolio-scoped diff;
    - SQLite online backup;
    - explicit typed confirmation;
    - transaction boundaries;
    - post-operation integrity and reconciliation checks.
9. Document the current supported topology and explicitly label Docker/Kubernetes/live as unsupported.

Required tests

- Table-driven tests for every mode/port/account/database combination.
- Test that missing IBC configuration fails before connecting.
- Test that all sanctioned entrypoints invoke the same validation.
- Test that live mode remains impossible even with contradictory environment variables.
- Test that maintenance utilities default to preview and cannot delete without the authorization sequence.

Done means

- No sanctioned command can connect to port 4001 or create a live executor.
- Dashboard and logs derive mode from the validated runtime contract.
- Paper and potential live state paths cannot collide.
- Destructive utilities cannot modify data by import or by default invocation.
- The supported-topology document matches executable behavior.

Rollback

Revert only to the previous paper launcher. Never restore destructive utility behavior.

PR 2 - Implement a reduce-only safety plane

Objective

Ensure kill switches and circuit breakers stop new exposure without blocking exits that strictly reduce existing exposure.

Problems addressed

- Kill-switch lock blocks stop-loss orders.
- All-order blocking conflates entry safety with emergency liquidation.
- Emergency shutdown cancels local stops without cancelling broker orders or flattening.
- Stop callbacks and state changes can diverge after failure.

Step-by-step work

1. Define an OrderIntent contract containing account, portfolio, symbol, side, quantity, current signed position, target signed position, reason, strategy, reduce-only flag, and idempotency key.
2. Implement a pure exposure-delta validator:
    - long SELL quantity cannot exceed the existing long position;
    - short BUY-to-cover cannot exceed the existing short position;
    - a reduce-only order cannot cross through zero;
    - an absent or uncertain position rejects the order.
3. Split gates into risk-increasing and risk-reducing policies.
4. Make kill switch, daily-loss limit, max-position limit, entry circuit breaker, and closed entry window reject only risk-increasing intents.
5. Keep broker disconnect, uncertain broker position, stale account state, and failed reconciliation as hard blocks even for automated exits; escalate these to an operator rather than guessing.
6. Route stop-loss and future flatten operations through the reduce-only validator.
7. Persist the reason and authorization decision for every rejected and accepted safety order.
8. Make emergency behavior explicit:
    - cancel open entry orders;

    - preserve or replace protective exit orders;
    - never claim flattening unless broker fills prove it.
9. Correct execution failure handling, including the .message versus .msg mismatch, so rejection details are never hidden by secondary exceptions.

Required tests

- Long reduction, full close, over-close, and reversal attempts.
- Short cover, full cover, over-cover, and reversal attempts.
- Stop execution while kill switch is active.
- Stop execution while daily limits or entry circuit breakers are active.
- Unknown broker position and stale reconciliation cases.
- Repeated identical safety intents proving idempotency.

Done means

- A gap-triggered kill switch cannot block a valid reduce-only exit.
- No reduce-only path can create or reverse exposure.
- Every decision has a durable reason and test coverage.

PR 3 - Establish the canonical market-data contract

Objective

Make every trading, stop, database, strategy, and dashboard consumer use validated, timezone-aware, session-correct market data.

Problems addressed

- RangeIndex values stored as timestamps.
- Database history overwritten on every refresh.
- Historical-bar strings not normalized.
- Regular-hours data used during extended-hours trading.
- Active runner bypasses authoritative validation.
- Stop monitor depends on periodically stale historical closes.
- Concurrent performance timers overwrite one another.

Step-by-step work

1. Define a versioned bar schema: symbol, exchange, timezone-aware timestamp, open, high, low, close, volume, session, source, retrieval time, adjustment state, and quality flags.
2. Parse IBKR dates at the subprocess boundary and reject ambiguous or timezone-naive values.
3. Set the normalized timestamp as the DataFrame index before returning data.
4. Validate OHLC ordering, finite values, nonnegative volume, monotonic order, duplicates, gaps, staleness, and session membership.

5. Store actual timestamps and use explicit upsert rules that preserve history.
6. Make regular versus extended-hours retrieval an explicit strategy/runtime choice.
7. Add an independent quote or real-time bar channel for protective monitoring.
8. Define degraded behavior for stale or missing data:
    - block new entries;
    - retain broker-native protection;
    - alert the operator;
    - never silently substitute the last regular-session close for an extended-hours decision.
9. Make performance timer identifiers unique per symbol and operation instance.
10. Add data lineage and freshness to dashboard/API responses.

Required tests

- DST transitions, market holidays, regular/extended session boundaries, and timezone conversions.
- Duplicate, reversed, NaN, infinite, zero-volume, and out-of-order bars.
- Database accumulation across repeated refreshes.
- Extended-hours fetch and decision tests.
- Stale protective feed behavior.
- Multi-symbol concurrent timer tests.

Done means

- Database timestamps match broker timestamps.
- Historical refreshes accumulate rather than replace numbered rows.
- No strategy or stop consumes unvalidated bars.
- Extended-hours state is explicit and tested.

PR 4 - Make financial state durable and database operations safe

Objective

Create a recoverable financial ledger with constrained schema, safe pooling, safe migrations, and verified backups.

Problems addressed

- Connection-pool replacement can deadlock and requeue closed connections.
- Financial tables lack foreign keys and value constraints.
- SQLite REAL is used for critical money values.
- Trades lack broker order/execution IDs and lifecycle fields.
- Formal multiuser migration is not wired; partial startup migrations swallow failures.
- Migration backup is not WAL-safe and may collapse account rows.
- Deployment and dashboard database settings disagree.

- No demonstrated restore-ready backup exists.

Step-by-step work

1. Select one authoritative database setting and require every runner, dashboard, utility, container, backup, and migration to use it.
2. Fix pool replacement so exactly one valid connection returns to the bounded queue and the internal pool inventory remains correct.
3. Add a schema-version table and explicit ordered migrations. Remove broad exception swallowing.
4. Add database constraints for:
    - valid sides and lifecycle states;
    - positive quantities and prices where required;
    - finite, bounded risk percentages;
    - portfolio references;
    - unique idempotency and broker execution identifiers.
5. Enable foreign-key enforcement for every connection.
6. Store monetary values in lossless integer minor units or validated decimal strings according to a documented convention.
7. Add append-only tables for order intent, broker order state, fills, commissions, reconciliation snapshots, safety events, and administrator actions.
8. Make position/account projections derived from confirmed fill events or updated transactionally with those events.
9. Use SQLite's online backup API or a verified exclusive checkpoint procedure.
10. Encrypt and rotate off-host backups without exposing broker credentials.
11. Add automated backup integrity checks and a clean-room restore test.
12. Create migration dry-run, row-count, checksum, and rollback reports.

Required tests

- Pool failure injection under concurrency.
- Constraint and foreign-key rejection tests.
- Duplicate broker callback/idempotency tests.
- WAL-active backup followed by clean restore.
- Interrupted migration at each step.
- Multiportfolio preservation tests.
- Deployment database path and volume persistence tests.

Done means

- A database exception cannot deadlock the runner.
- An invalid financial row cannot enter through normal application paths.
- Backups include WAL state and restore to an integrity-clean database.
- Every order and fill has durable correlation identifiers.

PR 5 - Add read-only broker reconciliation

Objective

Make broker truth visible and authoritative before any broker-writing capability exists.

Problems addressed

- Startup trusts only the local database.
- Database-load errors fail open to empty positions.
- Broker positions helper exists but is unused by the active runner.
- Open orders, completed orders, executions, cash, and commissions are not reconciled.
- Existing reconciliation documentation points to a missing script.

Step-by-step work

1. Extend the read-only broker adapter to fetch account identity/type, cash, buying power, signed positions, open orders, completed orders, executions, and commissions.
2. Normalize broker objects into versioned domain records.
3. Implement reconciliation at:
    - startup;
    - reconnect;
    - fixed periodic intervals;
    - before arming future live capability;
    - after any ambiguous order state in later phases.
4. Compare broker state with the append-only ledger and derived projections.
5. Classify differences as expected timing lag, recoverable missing event, duplicate event, account mismatch, quantity mismatch, cash mismatch, or unknown.
6. Quarantine trading on all unknown or material mismatches.
7. Provide an operator diff that never auto-deletes data.
8. Persist reconciliation snapshots and resolution actions.
9. Expose reconciliation age and status on the dashboard.

Required tests

- Clean startup reconciliation.
- Local missing fill recovered from broker execution.
- Duplicate callback.
- Wrong account, wrong quantity, stale cash, unknown order, and reconnect scenarios.
- Database unavailable or partially migrated.
- Operator resolution without destructive replacement.

Done means

- Startup cannot continue trading after local-state uncertainty.
- Every mismatch produces a durable, understandable diff.

- No reconciliation operation deletes history.

Phase B - Repair strategy evidence and execution policy

PR 6 - Rewrite backtesting and walk-forward validation

Objective

Create a deterministic, auditable backtest and replay system that can support strategy decisions without look-ahead or accounting distortion.

Problems addressed

- Returns are always calculated as zero.
- Signals execute at the same bar used to generate them.
- Commissions and spread are double-counted.
- Slippage is unseeded.
- Partial fills are not implemented.
- Exceptions are swallowed and final liquidation is incomplete.
- Metrics assume an inappropriate frequency.
- Walk-forward selection contaminates out-of-sample evidence.
- Critical backtesting modules have no direct tests.

Step-by-step work

1. Reset all engine state at the start of every run and validate sorted, nonempty input.
2. Separate observation time, decision time, submission time, and executable fill time.
3. Default to next-bar or event-realizable fills.
4. Define one cost model where spread, slippage, fees, commissions, borrowing, and market impact are counted exactly once.
5. Seed all stochastic behavior and record the seed in results.
6. Implement liquidity limits and genuine partial fills.
7. Use intrabar OHLC semantics for stop/take-profit triggers while documenting ambiguity resolution.
8. Support long and short accounting, dividends, splits, borrow costs, and delistings as needed by candidate strategies.
9. Calculate equity after fills and append final liquidation/mark-to-market economics.
10. Annualize metrics from actual sampling frequency and guard every zero denominator.
11. Fail the run on data/strategy exceptions unless an explicit, reported policy says otherwise.
12. Implement rolling or nested walk-forward validation with an untouched final holdout.
13. Produce a reproducibility manifest containing code SHA, data checksum, configuration, costs, seed, and model identifiers.

Required tests

- Hand-calculated golden portfolios.
- Look-ahead detection fixtures.
- Commission/spread charged once.
- Partial fill and volume constraints.
- Stop gaps and intrabar ambiguity.
- Empty input, all-error input, and final liquidation.
- Deterministic reruns and untouched holdout verification.

Done means

- Golden fixtures match exact hand calculations.
- Repeated runs with the same manifest are identical.
- No strategy receives launch approval from the old engine.

PR 7 - Unify risk, sizing, exits, and strategy contracts

Objective

Replace duplicate, drifting risk and strategy pathways with one authoritative decision-to-order contract.

Problems addressed

- Hardcoded 10% advanced position cap versus configured 2% cap.
- Symbol validation mistakenly uses the sector limit.
- Daily notional resets after restart.
- Nearest-share rounding can exceed limits.
- Missing market data fails some checks open.
- Existing/short positions are missing or wrong in advanced-risk state.
- Take-profit exists as metadata but not an authoritative lifecycle.
- Pairs legs are non-atomic and use synthetic model training/placeholders.
- Extracted runner modules, `core/engine.py`, and active runner duplicate behavior.

Step-by-step work

1. Select one active strategy interface and one active risk engine.
2. Define a versioned `Signal -> OrderIntent` contract containing confidence, expected horizon, source data version, sizing request, entry policy, stop policy, take-profit policy, and expiry.
3. Centralize all configuration loading and remove hardcoded risk values.
4. Enforce position, portfolio, sector, correlation, leverage, liquidity, buying-power, daily notional, churn, and duplicate limits in one place.
5. Floor quantities so calculated exposure never exceeds a cap.
6. Restore daily counters from the ledger on startup.

7. Synchronize long, short, and existing positions from reconciled state.
8. Define take-profit semantics as either broker bracket behavior or an explicitly tested exit policy.
9. Disable incomplete strategies by default, especially pairs, shorts, smart execution, AI discovery, and synthetic-trained selectors.
10. For future pairs support, validate both legs before submission and add combo/hedge, timeout, and compensating-unwind behavior.
11. Archive or clearly quarantine dead alternate engines and unused extracted runner modules until intentionally integrated.
12. Produce a strategy readiness card for every candidate strategy: data, parameters, risk rules, tests, backtest manifest, shadow results, and enabled environments.

Required tests

- Every risk limit across every enabled strategy.
- Restart persistence of daily counters.
- Long and short position accounting.
- Quantity flooring at boundary values.
- Missing data and missing sector/correlation inputs fail closed.
- Two-leg failures and compensating behavior before pairs can be enabled.

Done means

- A configured 2% position cap cannot produce or validate a larger intent.
- Only strategies with readiness cards can be enabled.
- One code path owns sizing, stops, take-profit, and risk validation.

Phase C - Secure remote access and make delivery trustworthy

PR 8 - Build the identity, authorization, and WebSocket security boundary

Objective

Permit safe read-only remote use without exposing broker credentials or granting every user administrative control.

Problems addressed

- Dashboard authentication is disabled locally and uses one shared credential when enabled.
- No active RBAC, MFA, portfolio ownership, or individual actor audit.
- Kill-switch reset/start/stop use ordinary dashboard credentials.
- Shared WebSocket token allows consumer-to-producer impersonation.

- Token is placed in HTML/query strings; null/missing origins are admitted.
- Basic auth can be exposed by direct published ports.
- Legacy unsalted SHA-256 password setup.
- Model signature enforcement uses inconsistent mode detection.

Step-by-step work

1. Introduce individual identities with strong password hashing and MFA.
2. Define roles such as viewer, portfolio operator, safety operator, and administrator.
3. Enforce portfolio-level authorization on every HTTP and WebSocket response.
4. Use secure server-side sessions or scoped, short-lived tokens with rotation and revocation.
5. Require reauthentication, a reason, and elevated permission for start, stop, kill-switch reset, and future live arming.
6. Consider two-person approval for live kill-switch reset.
7. Write administrative actions to append-only audit storage before applying the change where safety permits.
8. Replace shared producer trust with local IPC, mTLS, or a separate producer credential unavailable to dashboard consumers.
9. Use same-origin WSS through the TLS proxy; remove query-string tokens and reject null/missing origins for remote deployment.
10. Add message schemas, size limits, rate limits, subscription authorization, and portfolio filtering.
11. Move all secrets to approved environment/secret-manager injection and implement rotation procedures.
12. Make model signing mandatory from the canonical runtime contract and deserialize the exact verified bytes.
13. Add recursive log/config redaction and stop logging broker account identifiers except approved aliases.

Required tests

- Role and portfolio access matrix.
- Session revocation, expiration, MFA, and reauthentication.
- CSRF, CORS, origin, proxy, brute-force, and rate-limit tests.
- Consumer attempts producer impersonation.
- Query-token leakage and log-redaction tests.
- Unsigned/tampered model refusal in every production-like mode.

Done means

- Remote users are individually attributable and least-privileged.
- Dashboard consumers cannot publish runner events.
- Safety actions have elevated controls and durable audit.
- Mobile access may begin only as read-only.

PR 9 - Restore CI, packaging, dependency, and supply-chain truth

Objective

Make a green build mean the installable artifact, critical tests, security checks, and supported runtime all passed.

Problems addressed

- 551 current mypy errors and formatting/lint failures.
- False-green tests return booleans instead of asserting.
- Safety tests are ignored by default.
- Critical modules have zero coverage; app.py is excluded.
- Performance jobs are no-ops.
- Package metadata omits subpackages and runtime dependencies.
- Security scans do not always scan installed application dependencies.
- CI actions and container images use mutable references.
- Model/runtime dependency versions drift.

Step-by-step work

1. Choose one authoritative CI workflow and remove contradictory duplicates.
2. Package all robo_trader subpackages using package discovery.
3. Declare runtime dependencies and split dev, test, ML, and operations extras.
4. Create a reproducible lock with hashes for supported Python versions.
5. Build a wheel and test it in a clean environment rather than relying on checkout imports.
6. Convert return/print-based tests to assertions and remove unconditional success messages.
7. Stop ignoring the safety suite; rewrite it into deterministic tests.
8. Add direct tests for backtester, order lifecycle, reconciliation, stop safety, database failures, and deployment startup.
9. Include dashboard/API code in coverage and set risk-based coverage thresholds.
10. Resolve type errors in active safety paths first; maintain a shrinking, explicit debt budget for lower-risk legacy code.
11. Add real performance, leak, concurrency, and soak jobs.
12. Scan the resolved production dependency set and built image.
13. Pin third-party actions and images to reviewed SHAs/digests.
14. Generate SBOM and provenance, sign artifacts, and verify them during deployment.
15. Pin model-training and inference environments and reject incompatible serialized artifacts.

Required tests

- Clean wheel install and smoke test.
- Exact locked environment test matrix.

- Coverage gates on critical modules.
- Dependency and container vulnerability policy.
- Reproducible artifact/hash comparison.
- No pending asyncio tasks or leaked processes at test completion.

Done means

- Required CI is green with no placeholder jobs.
- A clean wheel contains every required module and dependency.
- Critical safety paths have meaningful assertion coverage.

PR 10 - Make telemetry, alerts, and the dashboard authoritative

Objective

Give operators a truthful, fresh, source-attributed view of the actual runner and broker state.

Problems addressed

- Dashboard panels contain hardcoded/mock metrics.
- Missing runner data is presented as healthy zero/default state.
- Dashboard process-local objects do not reflect runner state.
- Mode/status can be stale or contradictory.
- Webhook alerts are disabled in the current environment.
- Logs are tracked in Git and may contain sensitive trading telemetry.
- Mobile layout, accessibility, and polling behavior are weak.

Step-by-step work

1. Define versioned presentation contracts for runtime, broker, orders, fills, positions, risk, strategies, data freshness, reconciliation, protection, alerts, and build identity.
2. Include source, timestamp, age, portfolio, environment, and availability state in every response.
3. Persist runner telemetry to a shared durable store or event stream rather than accessing process-local objects.
4. Remove every hardcoded or estimated value presented as actual execution/risk state.
5. Render Unavailable, Stale, Degraded, Mock, and Zero distinctly.
6. Add non-dismissible mode/account/reconciliation/protection banners.
7. Expose last completed cycle, last successful broker query, last fill, data age, current kill-switch reason, and active broker protective orders.
8. Test email/SMS/pager/webhook alerts end-to-end and add a dead-man alert.
9. Add tamper-evident audit event access for authorized operators.
10. Remove tracked runtime logs from future commits and document safe history-cleaning as a separate user-approved operation if desired.
11. Use same-origin WSS, responsive breakpoints, accessible tabs, and active-tab/subscription-driven refresh.

12. Keep the initial mobile surface read-only.

Required tests

- API contract and freshness tests.
- Runner stopped, stale, DB error, broker disconnected, alert failed, and reconciliation mismatch states.
- Browser tests for desktop/mobile layouts and keyboard navigation.
- Verified human receipt of every critical alert class.

Done means

- No mock or unavailable value is displayed as real/healthy.
- Operators can determine mode, account, freshness, reconciliation, and protection at a glance.
- Critical alerts reach a human and are recorded.

Phase D - Add live capability behind a disabled gate

PR 11 - Implement one live IBKR order lifecycle adapter

Objective

Create a complete broker order state machine while keeping the capability disabled in all normal environments.

Problems addressed

- Dormant `LiveExecutor` schedules orders ambiguously and treats `Submitted` as filled.
- No active broker `place_order` implementation.
- No idempotent order reference or authoritative broker lifecycle.
- No partial fill, reject, cancel, replace, commission, reconnect, or unknown-state handling.
- Order state is updated from simulation rather than broker fills.

Step-by-step work

1. Define one asynchronous broker interface. Remove or quarantine synchronous wrappers that can outlive returned results.
2. Use deterministic client order IDs/order references tied to durable order intents.
3. Model states: created, pending submit, pre-submitted, submitted, partially filled, filled, cancel pending, cancelled, rejected, inactive, unknown, and reconciliation required.
4. Persist broker order ID, permanent ID, execution ID, timestamps, quantities, prices, commissions, warnings, and rejection details.
5. Update positions, cash, risk, and dashboard only from broker-confirmed fills.
6. Make retries idempotent and ambiguity-safe. A timeout must transition to unknown/reconcile, never automatic duplicate submission.

7. Add account allowlist and contract qualification.
8. Enforce market-hours, order-type, notional, buying power, shortability, and exchange/session constraints.
9. Implement cancel and replace with reconciliation after reconnect.
10. Cancel open entry orders during safety shutdown while preserving reduce-only protection.
11. Keep the entire adapter behind a disabled capability gate and use an IBKR test/paper account for integration tests.

Required tests

- Submit acknowledgement without fill.
- Partial fills and multiple callbacks.
- Reject, inactive, cancel, replace, timeout, reconnect, duplicate callbacks, and duplicate client IDs.
- Wrong account and contract ambiguity.
- Process crash between broker fill and local persistence.
- No position change before confirmed fill.

Done means

- `Submitted` is never treated as a fill.
- Ambiguous outcomes cannot produce automatic duplicate orders.
- Broker fills are the only source of financial state transitions.
- Capability remains disabled outside explicit integration tests.

PR 12 - Add broker-native protective orders and exit lifecycle

Objective

Ensure every live position has broker-resident protection that survives local failures.

Problems addressed

- In-memory synthetic stops disappear when process/host/Gateway fails.
- Stop prices can be stale and gap behavior is unsafe.
- Take-profit is not an authoritative execution feature.
- Emergency flattening and broker cancellation are incomplete.

Step-by-step work

1. Define supported bracket/OCO structures for long and short positions.
2. Implement IBKR parent/child transmit sequencing so protection cannot be accidentally left unsubmitted.
3. Establish stop-market versus stop-limit policy; default gap protection must not create an unfillable stale limit.
4. Implement broker-native take-profit where strategy policy calls for it.

5. Reconcile protective orders on startup, reconnect, fill, partial fill, quantity change, and cancel/replace.
6. Implement trailing-stop modification with rate limits and broker acknowledgement.
7. Define overnight, extended-hours, outside-RTH, and corporate-action behavior.
8. Prevent entry completion from being considered safe until expected protection is acknowledged at the broker.
9. Add operator-visible protection status and emergency cancel/flatten runbooks.
10. Retain synthetic monitoring only as an independent alarm, not the primary protection.

Required tests

- Parent fill before/after child acknowledgements.
- Partial fills and protective quantity updates.
- Gap through stop, reconnect, Gateway failure, process kill, and host restart.
- Manual broker-side cancellation detection and repair.
- OCO behavior and trailing modifications.

Done means

- Broker UI proves protection remains active with the RoboTrader processes stopped.
- Every reconciled live position has the required protection or trading is quarantined.

Phase E - Prove operations, then allow only a limited launch

PR 13 - Failure injection, restoration, and multi-week paper soak

Objective

Prove that the complete system behaves safely under realistic failures before enabling live capability.

Step-by-step work

1. Build a broker simulator covering submit, acknowledge, partial fill, fill, reject, cancel, duplicate callback, disconnect, and reconnect.
2. Run controlled failures for:
    - stale/malformed market data;
    - Gateway restart and 2FA delay;
    - network partition;
    - process kill and host reboot;
    - database locked/corrupt/full;
    - disk full and log growth;
    - alert provider failure;
    - duplicate executions;

    - missing or cancelled protective orders;
    - second-leg pairs failure if pairs remains planned.
3. Rehearse online backup and clean-machine restore.
4. Rehearse kill switch, cancel open entries, and manual flatten using the paper broker.
5. Verify reconciliation after every failure.
6. Conduct a multi-week paper/shadow soak using the exact release artifact and operational topology.
7. Track restarts, stale data, reconciliation mismatches, duplicate order attempts, pending tasks, DB latency, alert latency, and unbounded logs.
8. Produce a signed launch-readiness evidence package.

Done means

- Zero unexplained duplicate orders.
- Zero unresolved reconciliation drift.
- Every active paper position has expected protection behavior.
- Restore and alert drills succeed.
- Soak exits within agreed reliability and risk thresholds.

PR 14 - Deliver the selected remote/mobile/cloud topology

Objective

Support remote or cloud operation only after an explicit architecture decision and without creating multiple order writers.

Step-by-step work

1. Decide whether macOS + IBC remains the only order-writing topology.
2. If cloud is not required, remove misleading Kubernetes/live deployment artifacts and publish a secure remote read-only dashboard design.
3. If cloud is required, design:
    - IB Gateway ownership and interactive 2FA;
    - single active writer with lease and fencing;
    - encrypted persistent database and backups;
    - secret-manager integration;
    - private network and NetworkPolicy;
    - immutable signed images;
    - real liveness/readiness endpoints;
    - rolling deploy prevention for the order writer;
    - rollback and disaster recovery.
4. Make Docker paper mode boot and persist correctly before any production manifest.
5. Replace placeholder deployment jobs with real staging, smoke, health, approval, production, and rollback operations.

6. Release mobile as read-only first. Add remote mutating actions only after separate threat modeling and approval.

Done means

- Exactly one fenced order writer can exist.
- Health checks measure actual runner/broker/reconciliation readiness.
- Deployment and rollback have been exercised, not merely documented.
- Remote clients use strong identity and TLS without direct broker credential access.

PR 15 - Limited live canary and staged expansion

Objective

Enable the smallest reasonable real-money exposure only after all prior gates pass.

Preconditions

- PRs 1 through 14 are complete as applicable.
- All P0 and P1 audit findings are closed.
- Independent security and trading-safety review approves the evidence package.
- Reconciliation is clean.
- Broker-native protection is visible in IBKR.
- Human alerts, backup restore, and manual flatten drills pass.
- Multi-week paper/shadow soak passes.

Step-by-step work

1. Create separate live account configuration, credentials, database, logs, model artifacts, and deployment identity.
2. Require a manual arming ceremony with named operator, reason, build SHA, configuration fingerprint, account confirmation, and expiry time.
3. Start with:
    - tiny symbol allowlist;
    - one open position maximum;
    - very low absolute notional and daily-loss caps;
    - long-only simple orders;
    - no pairs, shorts, smart execution, AI discovery, extended-hours entries, or automatic strategy expansion.
4. Require broker-native stop protection before the entry is considered operationally complete.
5. Monitor every order and fill in real time with human acknowledgement.
6. Automatically disable new entries on any reconciliation, data, alert, protection, or process-health degradation.
7. Review after each trade and each day. Expansion requires a new approved stage, never an automatic threshold change.

8. Maintain a tested manual cancel/flatten path and a documented return-to-disabled procedure.

Done means

- Canary trades reconcile exactly with broker records.
- No safety or operational deviation is unexplained.
- Expansion is separately approved with evidence.

6. Launch gates

Gate A - Supervised local paper readiness

Required PRs: 1 through 5, plus relevant parts of 7.

Evidence required:

- Broker confirms paper account and read-only API.
- Paper state cannot collide with any future live state.
- Reduce-only exits pass all blocking-state tests.
- Data timestamps, freshness, and session semantics are correct.
- Startup fails closed on database or reconciliation uncertainty.
- Backups restore cleanly.
- No unsafeguarded destructive utility remains.

Gate B - Strategy evaluation readiness

Required PRs: 6 and 7.

Evidence required:

- Golden backtest accounting passes.
- No same-bar look-ahead.
- Costs are counted once.
- Results reproduce from a manifest.
- Candidate strategy has a readiness card and untouched holdout results.
- Risk and sizing caps are proven across all paths.

Gate C - Remote/mobile read-only readiness

Required PRs: 8 through 10.

Evidence required:

- Identity, MFA, least privilege, portfolio authorization, TLS, and revocation tests pass.
- WebSocket consumers cannot impersonate the producer.

- Mobile is read-only.
- Dashboard never presents stale/mock/unavailable values as healthy facts.
- Alerts reach a human.

Gate D - Live implementation readiness

Required PRs: 11 and 12.

Evidence required:

- Complete order lifecycle works in IBKR paper integration.
- Submitted is distinct from filled.
- Ambiguous timeouts reconcile without duplicate submission.
- Broker-native protection survives process and host failure.
- Broker truth drives all financial state.

Gate E - Live canary readiness

Required PRs: 13 through 15.

Evidence required:

- Failure drills, backup restore, alerts, and multi-week soak pass.
- Supported deployment topology is explicit and tested.
- Independent safety/security approval is recorded.
- Canary constraints and manual arming are active.

7. Cross-cutting problem register

Use these identifiers in issues and PR descriptions.

- RT-001: Active runtime always uses PaperExecutor; no coherent live path.
- RT-002: Dormant LiveExecutor has ambiguous async behavior and treats Submitted as success.
- RT-003: No broker-authoritative startup/reconnect reconciliation.
- RT-004: Kill-switch lock blocks reduce-only stop exits.
- RT-005: Stops are synthetic and periodically stale.
- RT-006: No broker-native bracket/OCO protection or authoritative take-profit.
- RT-007: Position limits drift between configuration and risk implementations.
- RT-008: Daily counters and advanced-risk state do not restore correctly.
- RT-009: Pairs execution is non-atomic and selector logic contains synthetic placeholders.
- RT-010: Market-data timestamps are stored as RangeIndex values.
- RT-011: Extended-hours decisions may consume regular-hours-only data.

- RT-012: Active data validation and performance telemetry are incomplete.
- RT-013: Backtest returns, costs, execution timing, and walk-forward evidence are invalid.
- RT-014: Destructive utilities can erase databases or positions.
- RT-015: Database pool error path can deadlock.
- RT-016: Financial schema lacks broker identifiers, strong constraints, and lossless values.
- RT-017: Migration and backup behavior is not WAL-safe.
- RT-018: Database paths differ across runner, dashboard, Compose, Kubernetes, and backup tools.
- RT-019: No verified automated off-host backup and restore program.
- RT-020: Shared dashboard credential lacks authorization and individual attribution.
- RT-021: Kill-switch reset/start/stop lack elevated approval and durable audit.
- RT-022: Shared WebSocket token allows producer impersonation.
- RT-023: Model signing and runtime-version enforcement can fail open.
- RT-024: Logging and Git history expose sensitive trading telemetry.
- RT-025: Tests have false-green patterns, ignored suites, and critical coverage gaps.
- RT-026: CI, packaging, dependency scanning, and supply-chain controls are not release-grade.
- RT-027: Docker/Kubernetes paths bypass preflight and do not persist the actual ledger.
- RT-028: Deployment workflows contain placeholder deploy, smoke, and rollback steps.
- RT-029: Dashboard contains hardcoded/mock or process-local operational values.
- RT-030: Mobile/cloud transport, accessibility, and topology are incomplete.
- RT-031: Duplicate unfinished engines and runner modules create architecture drift.
- RT-032: Documentation claims conflict with executable readiness.

8. Progress register

Update after each merge.

- Phase 0 CI truth gate: PR #83 merged on 2026-07-23 (7f5de0a)
- Phase 0 runtime-stability prerequisite: PR #81 merged on 2026-07-23 (b7e5005)
- PR 1: PR #82 is implementation-complete locally and awaiting hosted merge gates. Local evidence on 2026-07-23: 1,046 passed, 5 skipped, 42% total coverage; Black, isort, Flake8, Bandit, pip integrity, shell syntax, YAML parsing, and diff checks passed. Two Docker Compose render tests were skipped because Docker is unavailable on the local Mac and must pass in hosted CI. A two-phase review examined 11 initial findings: nine were confirmed and remediated, one dashboard lsof diagnostic was downgraded and remediated, and RT_STATE_NAMESPACE file-path isolation was safely deferred because changing the legacy paper kill-switch path could bypass the currently triggered state. The challenger then rejected three successive lifecycle designs until startup ordering,

the operator-facing Gateway CLI, concurrent launcher/recovery races, and lock ownership were all fail-closed. The final design acquires one kernel advisory lock, transfers it to the launcher with an inherited descriptor, validates that descriptor before runtime work, and prevents Gateway, dashboard, or runner descendants from retaining it. The final independent review passed. The active runner already rejects backtest mode; separate non-paper safety state remains required before that mode may use shared risk components.

- PR 2: Not started
- PR 3: Not started
- PR 4: Not started
- PR 5: Not started
- PR 6: Not started
- PR 7: Not started
- PR 8: Not started
- PR 9: Not started
- PR 10: Not started
- PR 11: Not started
- PR 12: Not started
- PR 13: Not started
- PR 14: Not started
- PR 15: Not started

9. Standard PR evidence checklist

Every safety-relevant PR must include:

- problem identifiers addressed;
- explicit non-goals;
- design or threat model where appropriate;
- migration and rollback behavior;
- unit, integration, and failure-injection tests;
- exact commands and results;
- database backup and restore implications;
- paper/live separation implications;
- security and credential implications;
- operator/dashboard implications;
- updated runbook and documentation;
- before/after screenshots for operator-facing changes;
- reviewer sign-off from trading safety and security/data integrity;
- confirmation that no user trading data was deleted.

10. Recommended verification commands

Commands must be adjusted as the CI contract evolves, but the starting set is:

```
.venv/bin/python -m pytest tests/ -q
.venv/bin/python -m pytest tests/ -q --cov=robo_trader --cov-report=term
.venv/bin/python -m mypy robo_trader
.venv/bin/python -m black --check .
.venv/bin/python -m flake8
.venv/bin/python -m bandit -r robo_trader scripts -ll
.venv/bin/python -m pip check
python3 scripts/preflight_check.py --verbose
python3 scripts/gateway_manager.py status
lsof -nP -iTCP:4002 -sTCP:LISTEN
lsof -nP -iTCP:4002 -sTCP:CLOSE_WAIT
```

Never use a test command that can access the production database unless test isolation has been verified for that exact command.

11. Final launch decision template

The launch approver must answer all of the following with evidence:

1. Which exact build, configuration fingerprint, model artifacts, account, database, and topology are being armed?
2. Is broker reconciliation clean now?
3. Are broker-native protective orders verified?
4. Are all risk limits loaded from the canonical configuration and displayed correctly?
5. Can reduce-only exits execute while entry trading is blocked?
6. Did backup restoration and manual cancel/flatten drills pass?
7. Are human alerts working?
8. Did the required paper/shadow soak pass without unexplained drift?
9. Are identity, authorization, WebSocket, and audit controls active?
10. Is the live canary restricted to the approved account, symbols, position count, notional, and expiry?

If any answer is no, unknown, stale, or based only on documentation rather than observed evidence, live trading remains disabled.